Árvores de Steiner com Rotulação Mínima

Implementação de Heurística

Raul Ramalho Lucena, Victor Xavier Bastos

16 de Outubro de 2026, Natal

Resumo

O problema da Árvore de Steiner com Rotulação Mínima (MLSteiner) consiste em conectar um subconjunto de vértices básicos de um grafo com arestas rotuladas usando uma árvore que empregue o menor número possível de rótulos distintos. O problema é NP-difícil, mesmo com apenas dois vértices básicos. Este trabalho implementa a heurística gulosa MVCA-Steiner, baseada na proposta de Cerulli et al. (2006), com um critério de desempate definido pelos autores e uma fase final de eliminação de rótulos redundantes. Nos experimentos, com 720 instâncias, o algortimo construído seguindo as instruções de Cerulli et al. (2006), a heurística atingiu o valor de referência em 82% das 480 instâncias com até 100 vértices, com desvio relativo médio de 4,9% e no máximo dois rótulos acima dele, e resolveu instâncias com 500 vértices em menos de um segundo.

Palavras-chave: Árvore de Steiner. Rotulação mínima. Heurística. Grafos.

1 Introdução

O problema da árvore de Steiner em grafos pede o subgrafo de custo mínimo que conecte um conjunto de vértices terminais, podendo passar por vértices intermediários. É um dos problemas centrais da otimização combinatória: está entre os problemas NP-completos de Karp (1972) e tem aplicações em projeto de redes de telecomunicações, transportes e circuitos integrados (Hwang; Richards; Winter, 1992). Em muitas dessas aplicações, porém, o custo relevante não é a soma dos pesos das arestas, mas o número de categorias diferentes envolvidas. Cada enlace pode pertencer a uma operadora, a um modo de transporte ou a uma tecnologia, e cada categoria adicional implica um contrato, uma interface ou um equipamento a mais. Esse cenário é modelado por grafos com rótulos (cores) nas arestas.

O primeiro problema desse tipo a ser estudado foi o da Árvore Geradora com Rotulação Mínima (MLST), proposto por Chang e Leu (1997): conectar todos os vértices de um grafo com uma árvore que use o menor número de rótulos. Os autores mostraram que o problema é NP-difícil, ou seja, que não se conhece um algoritmo eficiente que o resolva de forma exata, e propuseram uma heurística gulosa chamada MVCA. A ideia é simples: começar sem nenhum rótulo e, a cada passo, escolher o rótulo que mais reduz o número de partes desconectadas do grafo, até que tudo esteja conectado. Krumke e Wirth (1998) ajustaram essa heurística e provaram que ela nunca usa mais do que cerca de $2 \ln n$ vezes o número ótimo de rótulos, sendo n o número de vértices. Wan, Chen e Xu (2002) melhoraram esse limite para $\ln(n-1) + 1$.

Cerulli et al. (2006) estenderam o problema para o caso em que não é preciso conectar todos os vértices, mas apenas um subconjunto deles, chamados vértices básicos. Surgiu assim o problema da Árvore de Steiner com Rotulação Mínima (MLSteiner), tema deste trabalho. Para resolvê-lo, os autores adaptaram o MVCA: em vez de contar todas as partes desconectadas do grafo, a heurística passa a contar apenas as que contêm vértices básicos. Essa versão gulosa serviu de base para várias metaheurísticas, das quais o *Pilot Method* teve os melhores resultados. Depois, Consoli et al. (2009) propuseram métodos como GRASP e VNS (busca em vizinhança variável), que também usam a heurística gulosa como etapa de construção e obtiveram soluções ainda melhores.

Como todos esses métodos dependem da heurística gulosa, ela é o ponto de partida natural para o estudo do problema. Este trabalho implementa essa heurística, aqui chamada MVCA-Steiner. O trabalho analisa o tempo de execução e a qualidade da heurística e a testa em instâncias criadas por um gerador baseado nos trabalhos anteriores.

2 Desenvolvimento

2.1 Descrição do problema

Seja $G = (V, E)$ um grafo não direcionado e conexo, com $n = |V|$ vértices e $m = |E|$ arestas. Cada aresta e recebe um rótulo $\lambda(e)$ de um conjunto L com $\ell = |L|$ rótulos, arestas diferentes podem ter o mesmo rótulo. É dado também um conjunto $Q \subseteq V$ de q vértices básicos. Os demais vértices, chamados vértices de Steiner, podem ser usados ou não.

O problema MLSteiner consiste em encontrar uma árvore, subgrafo de G, que contenha todos os vértices de Q e cujas arestas usem o menor número possível de rótulos distintos.

É mais fácil trabalhar com conjuntos de rótulos do que com árvores. Para $C \subseteq L$, seja $G[C]$ o subgrafo formado pelas arestas cujos rótulos estão em C. O conjunto C é *viável* se todos os vértices básicos ficam na mesma componente conexa de $G[C]$. Qualquer árvore que conecte Q dentro de $G[C]$ usa no máximo $|C|$ rótulos, por isso, o problema se resume a encontrar o menor conjunto viável C, e a árvore é obtida depois em tempo linear. Chama-se *componente de Steiner* uma componente conexa de $G[C]$ que contém ao menos um vértice básico, C é viável quando existe exatamente uma componente de Steiner.

2.2 Complexidade do problema

O MLSteiner é NP-difícil, pois contém dois problemas NP-difíceis como casos particulares:

- quando $Q = V$, ele se torna o MLST, provado NP-difícil por Chang e Leu (1997), por redução da cobertura de conjuntos, e por Broersma e Li (1997), por redução do conjunto dominante mínimo;
- quando $q = 2$, a árvore ótima é um caminho, e o problema se torna o do caminho s–t com rotulação mínima, também NP-difícil (Hassin, Monnot, Segev, 2007).

Quanto à aproximação, a situação é pior que a do MLST. Hassin, Monnot e Segev (2007) mostraram que o caminho com rotulação mínima não pode ser aproximado por um fator $\log^k n$, para nenhum k fixo, a menos que P = NP, e Zhang et al. (2011) fortaleceram esse resultado. Como esse problema é um caso particular do MLSteiner, nenhum algoritmo polinomial garante razão polilogarítmica para o MLSteiner. Uma consequência é que o limite $\ln(q-1)+1$, enunciado por Cerulli et al. (2006) para sua heurística gulosa por analogia com o MLST, não pode valer em geral: para $q = 2$ ele daria razão 1, o que resolveria um problema NP-difícil em tempo polinomial.

2.3 Heurísticas e algoritmos aproximativos da literatura

- **MVCA** (Chang, Leu, 1997, Krumke, Wirth, 1998): heurística gulosa para o MLST que adiciona, a cada passo, o rótulo que minimiza o número de componentes conexas. Tem razão de aproximação $\ln(n-1)+1$ (Wan; Chen; Xu, 2002).
- **Gulosa de Cerulli et al.** (Cerulli et al. (2006),): adapta o MVCA ao MLSteiner contando apenas as componentes de Steiner. No mesmo trabalho são comparados com outros algoritmos como o VNS, *simulated annealing*, busca tabu reativa e o

Pilot Method, que obteve os melhores resultados. O *Pilot Method* testa cada rótulo candidato completando a solução com a gulosa e fica com o que leva à menor solução.

- **Consoli et al.** (Consoli et al., 2009, 2010): GRASP, VNS, uma VNS híbrida (GS-VNS) e otimização por enxame de partículas discreta (DPSO). A VNS foi a melhor e encontrou o ótimo em todas as instâncias resolvidas pelo método exato dos autores.
- **Dehouche** (Dehouche, 2018): algoritmos genéticos "devolucionários" baseados em uma formulação de programação inteira.

Não há na literatura consultada algoritmo com garantia de aproximação para o MLSteiner.

2.4 Bancos de instâncias e links

Os trabalhos de referência usam instâncias aleatórias definidas por quatro parâmetros: número de vértices n, número de rótulos ℓ, densidade d e número de vértices básicos q, com dez instâncias por combinação. Cerulli et al. (2006) usaram $n \in \{50, 100\}$, com dados disponíveis apenas sob pedido aos autores; Consoli et al. (2009) usaram $n \in \{100, 500\}$, com dados publicados em uma página que hoje está fora do ar. Como nenhum desses bancos está disponível, foi implementado um gerador próprio (Seção 2.7). Links úteis:

- artigo que define o problema: https://doi.org/10.26636/jtit.2006.4.396;
- relatório técnico de Consoli et al.: http://bura.brunel.ac.uk/handle/2438/1337;
- tese de S. Consoli: http://bura.brunel.ac.uk/handle/2438/3061;
- Dehouche (2018): https://arxiv.org/abs/2004.10048.

2.5 Heurística implementada: MVCA-Steiner

A heurística implementada segue a gulosa de Cerulli et al. (2006), com duas modificações (Algoritmo 1):

1. **Desempate:** em grafos esparsos, é comum que nenhum rótulo reduza o número de componentes de Steiner, e a escolha original fica a cargo de um desempate arbitrário. O desempate proposto prefere o rótulo que absorve mais componentes sem vértices básicos para dentro das componentes de Steiner, ou seja, que "estende" essas componentes em direção às outras, persistindo o empate, escolhe o rótulo que realiza mais uniões.
2. **Eliminação:** ao final, tenta-se remover cada rótulo da solução, do menos para o mais frequente, mantendo a viabilidade.

Depois disso, a árvore é extraída por uma busca em largura no subgrafo $G[C]$, e as folhas que não são vértices básicos são removidas repetidamente. A implementação foi feita em Python, usando a estrutura *union-find* para manter as componentes conexas.

Algoritmo 1: MVCA-Steiner

Entrada : grafo $G = (V, E)$, rótulos $\lambda : E \to L$, vértices básicos $Q \subseteq V$
Saída : árvore T que conecta Q e seu conjunto de rótulos C

$C \leftarrow \emptyset$;
enquanto $G[C]$ *tem mais de uma componente de Steiner* **faça**
  para cada *rótulo* $c \in L \setminus C$ **faça**
    $s_c \leftarrow$ nº de componentes de Steiner de $G[C \cup \{c\}]$;
    $a_c \leftarrow$ nº de componentes sem básicos absorvidas por componentes de Steiner;
    $u_c \leftarrow$ nº total de uniões realizadas pelas arestas de rótulo c;
  $c^* \leftarrow$ rótulo com menor s_c; em caso de empate, maior a_c e depois maior u_c;
  $C \leftarrow C \cup \{c^*\}$;
para cada $c \in C$, *em ordem crescente de frequência* **faça**
  se $C \setminus \{c\}$ *é viável* **então** $C \leftarrow C \setminus \{c\}$;
$T \leftarrow$ árvore de busca em largura de $G[C]$, sem folhas que não sejam básicas;
retorne T, C

2.6 Análise do algoritmo

Complexidade de tempo. Seja k o número de iterações do laço principal; como cada iteração adiciona um rótulo e une ao menos duas componentes, $k \leq \min(\ell,\, n-1)$. Em cada iteração, todas as arestas dos rótulos ainda não usados são examinadas uma vez com operações de *union-find*, cujo custo por operação é tratado aqui como constante, o que dá $O(m)$ por iteração. Contar as componentes de Steiner custa $O(q)$ por rótulo, isto é, $O(\ell q)$ por iteração. A eliminação faz no máximo k testes de viabilidade de custo $O(m)$, e a extração da árvore custa $O(n+m)$. O tempo total é

$$O\big(k\,(m + \ell q)\big),$$

Limite de aproximação. Como todos os rótulos da solução final aparecem na árvore, que tem no máximo $n-1$ arestas, vale a garantia $|C| \leq \min(\ell,\, n-1) \cdot \mathrm{OPT}$. Esse limite não pode ser muito melhorado. Considere dois vértices básicos s e t ligados por um caminho curto $s - x - t$, com um rótulo diferente em cada aresta, e por um caminho longo de k arestas, também com um rótulo diferente em cada uma, o primeiro rótulo do caminho longo também aparece em duas arestas pendentes. A solução ótima usa 2 rótulos (o caminho curto). Porém, nenhum rótulo sozinho reduz o número de componentes de Steiner, e o desempate faz a heurística escolher, um a um, os k rótulos do caminho longo, que depois não podem ser eliminados. Como a instância tem $n = 3k$ vértices, a razão de aproximação é $\Theta(n)$ no pior caso. Esse comportamento foi confirmado executando o código para $k \in \{3, 5, 10, 20\}$.

2.7 Gerador de instâncias

O gerador segue o esquema de Cerulli et al. (2006) e Consoli et al. (2009). Dados (n, ℓ, d, q) e uma semente aleatória:

1. calcula-se o número de arestas $m = d\,n(n-1)/2$;

2. gera-se uma árvore geradora aleatória, o que garante que o grafo seja conexo;
3. sorteiam-se as demais arestas uniformemente, sem repetição, até completar m;
4. cada aresta recebe um rótulo sorteado uniformemente entre os ℓ rótulos;
5. sorteiam-se os q vértices básicos, sem reposição.

Foram usadas as classes $n \in \{50, 100, 500\}$, $\ell \in \{0{,}25n;\ 0{,}5n;\ n;\ 1{,}25n\}$, $d \in \{0{,}8;\ 0{,}5;\ 0{,}2\}$ e $q \in \{0{,}2n;\ 0{,}4n\}$, com dez instâncias por classe, totalizando 72 classes e 720 instâncias.

2.8 Resultados

Foram comparados, nas mesmas instâncias: o MVCA-Steiner (MVCA-S), a gulosa com o desempate original de Cerulli et al. (2006) (Cer.), o MVCA-S sem a fase de eliminação (s/elim.), e o *Pilot Method* (Pilot). Para $n \leq 100$, o valor de referência (OPT) foi obtido por um modelo de programação inteira resolvido com o OR-Tools CP-SAT, com limite de tempo, complementado por um método de geração de cortes. O ótimo foi provado em 440 das 480 instâncias, nas demais, OPT é a melhor solução conhecida e a classe é marcada com †. As tabelas mostram médias de dez instâncias por classe, "#ót." é o número de instâncias em que o MVCA-S atingiu OPT e "ms" é o tempo médio do MVCA-S em milissegundos.

Qualidade. Nas 480 instâncias com $n \leq 100$, o MVCA-S atingiu o valor de referência em 393 casos (81,9%). Nos 87 demais, ficou um rótulo acima em 85 instâncias e dois rótulos acima em duas. Desses erros, 55 ocorreram nas classes mais esparsas ($d = 0{,}2$). O desvio relativo médio foi de 4,9%. O *Pilot Method* atingiu o valor de referência em 476 instâncias (99,2%), mas executa a gulosa centenas de vezes.

Desempate e eliminação. Nas 720 instâncias, o desempate proposto deu solução melhor que o desempate original em 58 instâncias e pior em 19. A fase de eliminação reduziu a solução em 13 instâncias.

Tempo. O MVCA-S levou em média 6,9 ms com $n = 100$ e 443 ms com $n = 500$ (máximo de 906 ms, em grafos com cerca de 10^5 arestas). O *Pilot Method* levou em média 1,43 s com $n = 100$, cerca de 200 vezes mais.

Comparação com a literatura. A Tabela 1 compara as somas das médias com as publicadas. Como as instâncias originais não estão disponíveis, a comparação é entre amostras diferentes do mesmo tipo de gerador e deve ser lida com cautela. A diferença relativa entre a gulosa e o *Pilot Method*é parecida nos dois trabalhos, o que indica que a implementação reproduz o comportamento publicado. Já os valores absolutos das nossas instâncias, inclusive os ótimos, são maiores, sobretudo nas classes esparsas e com $n = 500$, o que sugere que o gerador original difere em algum detalhe não documentado. Por isso, a comparação principal é a feita nas mesmas instâncias (Tabela 1)

Discussão. Em grafos densos e com poucos rótulos, um ou dois rótulos já conectam todos os vértices básicos e a gulosa é quase sempre ótima. A dificuldade cresce com o número de rótulos, com o número de vértices básicos e, principalmente, com a esparsidade: em grafos esparsos os vértices básicos só se conectam passando por vértices de Steiner, muitos rótulos empatam no critério principal e a escolha gulosa erra com mais frequência. É nesses casos que o *Pilot Method*, que olha à frente antes de escolher, compensa seu custo maior.

Tabela 1 – Resultados para $n = 50$ (médias de 10 instâncias por classe).

q	ℓ	d	OPT	MVCA-S	Cer.	s/elim.	Pilot	#ót.	ms
10	12	0,8	1,0	1,0	1,0	1,0	1,0	10	1,2
		0,5	1,0	1,0	1,0	1,0	1,0	10	0,6
		0,2	2,2	2,2	2,3	2,2	2,2	10	0,7
	25	0,8	1,8	1,8	1,8	1,8	1,8	10	2,2
		0,5	2,0	2,0	2,0	2,0	2,0	10	1,6
		0,2	3,5	3,7	3,9	3,7	3,5	8	0,8
	50	0,8	2,0	2,0	2,0	2,0	2,0	10	2,3
		0,5	3,0	3,1	3,0	3,1	3,0	9	1,5
		0,2	4,9	5,3	5,6	5,3	4,9	6	1,2
	62	0,8	2,0	2,3	2,3	2,3	2,0	7	1,7
		0,5	3,0	3,1	3,5	3,1	3,0	9	1,5
		0,2	4,9	5,3	5,7	5,4	5,0	6	1,3
Soma ($q = 10$)			31,3	32,8	34,1	32,9	31,4	105	
20	12	0,8	1,0	1,0	1,0	1,0	1,0	10	0,9
		0,5	1,7	1,7	1,7	1,7	1,7	10	0,8
		0,2	2,9	2,9	2,9	2,9	2,9	10	0,6
	25	0,8	1,9	1,9	1,9	1,9	1,9	10	1,4
		0,5	2,0	2,0	2,0	2,0	2,0	10	1,0
		0,2	3,9	4,0	4,3	4,1	3,9	9	0,9
	50	0,8	2,3	2,7	2,7	2,7	2,3	6	2,1
		0,5	3,0	3,3	3,5	3,4	3,0	7	1,8
		0,2	6,0	6,6	6,6	6,7	6,0	4	1,7
	62	0,8	2,9	2,9	3,0	2,9	2,9	10	2,3
		0,5	3,8	3,9	3,9	3,9	3,8	9	2,1
		0,2	$6{,}4^{\dagger}$	7,1	7,0	7,4	6,6	3	1,9
Soma ($q = 20$)			37,8	40,0	40,5	40,6	38,0	98	

3 Conclusão

Este trabalho estudou o problema da Árvore de Steiner com Rotulação Mínima. Na parte teórica, foram reunidos os resultados de complexidade: o problema é NP-difícil mesmo com dois vértices básicos e não admite aproximação polilogarítmica, a menos que P = NP. Com isso, o limite logarítmico enunciado na literatura para a heurística gulosa não pode valer em geral, e um exemplo simples mostra que a razão da gulosa é $\Theta(n)$ no pior caso.

Na parte prática, foi implementada a heurística MVCA-Steiner, com custo $O(k\,(m+\ell q))$, um critério de desempate e uma fase de eliminação de rótulos, além de um gerador de instâncias. Nos testes, a heurística atingiu o valor de referência em 82% das instâncias com até 100 vértices, ficou no máximo dois rótulos acima dele e resolveu instâncias com 500 vértices em menos de um segundo. O desempate proposto melhorou a gulosa original em 58 instâncias.

4 Referências

BROERSMA, H.; LI, X. Spanning trees with many or few colors in edge-colored graphs. **Discussiones Mathematicae Graph Theory**, v. 17, n. 2, p. 259-269, 1997.

CERULLI, R. et al. Extensions of the minimum labelling spanning tree problem. **Journal of Telecommunications and Information Technology**, n. 4, p. 39-45, 2006.

CHANG, R.-S.; LEU, S.-J. The minimum labeling spanning trees. **Information Processing Letters**, v. 63, n. 5, p. 277-282, 1997.

CONSOLI, S. et al. Variable neighbourhood search for the minimum labelling Steiner tree problem. **Annals of Operations Research**, v. 172, n. 1, p. 71-96, 2009.

CONSOLI, S. et al. Discrete particle swarm optimization for the minimum labelling Steiner tree problem. **Natural Computing**, v. 9, n. 1, p. 29-46, 2010.

DEHOUCHE, N. Devolutionary genetic algorithms with application to the minimum labeling Steiner tree problem. **Evolving Systems**, v. 9, n. 2, p. 157-168, 2018.

HASSIN, R.; MONNOT, J.; SEGEV, D. Approximation algorithms and hardness results for labeled connectivity problems. **Journal of Combinatorial Optimization**, v. 14, n. 4, p. 437-453, 2007.

HWANG, F. K.; RICHARDS, D. S.; WINTER, P. **The Steiner tree problem**. Amsterdam: North-Holland, 1992.

KARP, R. M. Reducibility among combinatorial problems. In: MILLER, R. E.; THATCHER, J. W. (ed.). **Complexity of computer computations**. New York: Plenum Press, 1972. p. 85-103.

KRUMKE, S. O.; WIRTH, H.-C. On the minimum label spanning tree problem. **Information Processing Letters**, v. 66, n. 2, p. 81-85, 1998.

WAN, Y.; CHEN, G.; XU, Y. A note on the minimum label spanning tree. **Information Processing Letters**, v. 84, n. 2, p. 99-101, 2002.

ZHANG, P. et al. Approximation and hardness results for label cut and related problems. **Journal of Combinatorial Optimization**, v. 21, n. 2, p. 192-208, 2011.